

SIMATS ENGINEERING ASSESSMENT TOOLS

COURSE NAME: Theory of Computation

COURSE CODE: CSA1304

COURSE OUTCOME COVERED

CO3:Construct regular expressions, and use the pumping lemma and closure properties to analyze regular languages. (BL : 3)

Assessment Tool Weightage: 20%

QUESTIONS: 5
Duration: 1 Hour

Total Marks:25

CASE STUDY

Code of Conduct:

*I _Pusalapati Chandu(192372119)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: *Pchandu*

SIMATS ENGINEERING ASSESSMENT TOOLS

1. Designing a Grammar for Arithmetic Expressions

A compiler development team is creating a parser for a calculator application that supports addition, subtraction, multiplication, division, and parentheses. The initial grammar produces ambiguity for expressions such as $a+b*c$.

Case Study Question:

Develop an optimized context-free grammar (CFG) that correctly enforces operator precedence and associativity for the given arithmetic expressions. Explain how your grammar removes ambiguity and supports efficient parsing.

2. Mini Programming Language Parser Development

A startup is designing a small scripting language for automating IoT devices. The language supports variable declarations, conditional statements, and loops. However, the parser frequently reports syntax conflicts during compilation.

Case Study Question:

Construct a CFG for the scripting language constructs below and optimize it to eliminate left recursion and reduce parsing conflicts. Demonstrate how the optimized grammar improves parser efficiency.

3. Grammar Optimization for SQL Query Validation

A database company is developing a query validation tool for simplified SQL commands such as SELECT, FROM, and WHERE. The original grammar contains redundant productions that increase parsing time.

Case Study Question:

Develop a CFG for simplified SQL query syntax and optimize the grammar by removing unnecessary productions and ambiguity. Discuss how the optimized grammar helps in faster syntax analysis.

4. Natural Language Processing Chatbot

A chatbot system processes simple English sentences such as:

Subject + Verb + Object

Example: "Students learn TOC."

The initial CFG incorrectly accepts grammatically invalid sentences.

Case Study Question:

Design a CFG for the chatbot sentence structure and optimize it to improve language validation accuracy. Explain how the grammar ensures correct sentence formation and rejects invalid inputs.

5. Parser Design for HTML-like Markup Language

A web editor company is building a parser for a simplified markup language with opening and closing tags. The existing grammar fails to validate nested tags correctly.

Case Study Question:

Develop and optimize a CFG for validating properly nested tags in the markup language. Show how the grammar supports recursive nesting and improves parsing reliability.

SIMATS ENGINEERING ASSESSMENT TOOLS

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

Question 1 — Grammar for Arithmetic Expressions

Step 1: What is actually going wrong

The naive grammar for a calculator usually looks like this: $E \rightarrow E+E \mid E-E \mid E * E \mid E / E \mid (E) \mid \text{id}$. It looks fine at first glance, but it lets a single string be built in more than one way. Take $a+b*c$. One derivation groups it as $(a+b)*c$ and another groups it as $a+(b*c)$. Both are legal under the grammar, so the grammar is ambiguous — and an ambiguous grammar cannot tell a compiler which interpretation is 'correct'. Since multiplication should bind tighter than addition, only the second grouping is mathematically valid, but the grammar itself has no notion of 'tighter binding'.

Step 2: Fix it by layering the grammar by precedence

The standard fix is to introduce one non-terminal per precedence level, from lowest precedence at the top down to highest precedence at the bottom, with parentheses/identifiers as the innermost (highest priority) level:

SIMATS ENGINEERING ASSESSMENT TOOLS

$E \rightarrow E + T \mid E - T \mid T$	(E = expression: handles + and -)
$T \rightarrow T * F \mid T / F \mid F$	(T = term: handles * and /)
$F \rightarrow (E) \mid id$	(F = factor: parentheses or a value)

Step 3: Why this removes the ambiguity

Because * and / only ever appear inside T, and + and - only ever appear inside E, the grammar forces multiplication/division to be parsed as a self-contained sub-tree before it can be combined with addition/subtraction. For $a+b*c$, the only possible derivation is $E \rightarrow E+T \rightarrow T+T \rightarrow F+T \rightarrow id+T \rightarrow id+T*F \rightarrow id+F*F \rightarrow id+id*id$, which corresponds exactly to $a+(b*c)$. There is no second derivation available, so the ambiguity is gone by construction rather than by adding extra rules to 'break ties'.

Left-recursion in $E \rightarrow E+T$ and $T \rightarrow T*F$ is intentional here, not a mistake — it is what makes the grammar left-associative, so $a-b-c$ is correctly parsed as $(a-b)-c$ and not $a-(b-c)$. If the language instead needed right-associativity (like exponentiation, $a^b^c = a^{(b^c)}$), the same non-terminal would be written right-recursively instead: $F \rightarrow id \wedge F \mid id$.

Step 4: Why this helps the compiler team in practice

- Deterministic parsing: a recursive-descent or LL parser can now decide which rule to apply just by looking at the next token, with no backtracking.
- Correct precedence 'for free': the compiler never needs a separate precedence table because the grammar's structure already encodes it.
- Extensible: adding exponentiation later just means inserting one more layer ($E \rightarrow T, T \rightarrow F, F \rightarrow G, G \rightarrow G^{\wedge}F \mid id$) without touching the rules above it.

Question 2 — Mini Scripting Language for IoT Automation

Step 1: What 'syntax conflicts' usually means here

For a small language with variable declarations, if/else, and loops, syntax conflicts almost always trace back to one of two problems: (a) left recursion in the expression/statement rules, which makes a top-down parser recurse forever without consuming any input, and (b) the classic 'dangling else' ambiguity, where the parser can't tell which if an else belongs to in nested conditionals.

Step 2: Build the grammar in layers (declarations, statements, expressions)

Program	$\rightarrow$	StmtList
StmtList	$\rightarrow$	Stmt StmtList $\mid \epsilon$
Stmt	$\rightarrow$	VarDecl $\mid$ AssignStmt $\mid$ IfStmt $\mid$ LoopStmt
VarDecl	$\rightarrow$	'var' ID ':' Type ';'
Type	$\rightarrow$	'int' $\mid$ 'bool' $\mid$ 'float'
AssignStmt	$\rightarrow$	ID '=' Expr ';'

SIMATS ENGINEERING ASSESSMENT TOOLS

```
IfStmt      → 'if' '(' Expr ')' '{' StmtList '}' ElsePart
ElsePart    → 'else' '{' StmtList '}' | ε
LoopStmt    → 'while' '(' Expr ')' '{' StmtList '}'
```

Notice ElsePart is attached directly to the nearest, most recently opened IfStmt because it appears immediately after that if's own closing brace in the same production — the grammar never lets an else 'float free' looking for an if, so the dangling-else conflict simply cannot arise.

Step 3: Eliminate left recursion in the expression grammar

The Expr rules reuse the same layered idea as Question 1 (Expr for + / −, Term for * / and comparisons/booleans layered below), but suppose the team's first draft was left-recursive, e.g. $\text{Expr} \rightarrow \text{Expr} + \text{Term} \mid \text{Term}$. The general elimination rule is:

```
If:      A → A α | β      (β does not itself start with A)
Rewrite: A → β A'
         A' → α A' | ε
```

Applying this once to $\text{Expr} \rightarrow \text{Expr} + \text{Term} \mid \text{Term}$ gives $\text{Expr} \rightarrow \text{Term Expr}'$, $\text{Expr}' \rightarrow + \text{Term Expr}' \mid \epsilon$ — the language recognised is identical, but a top-down parser now consumes the Term first and only recurses on the ϵ -terminated tail, so it always makes progress through the input instead of looping.

Step 4: Why the optimized grammar is more efficient

- No left recursion → safe for a simple, fast recursive-descent parser (no need for a heavier LR/LALR engine just to compile IoT scripts).
- No dangling-else ambiguity → the compiler never has to insert an arbitrary rule ('else binds to nearest if') as a patch; it's structurally impossible.
- Each Stmt alternative starts with a distinct keyword ('var', identifier, 'if', 'while'), so the parser can pick the right rule by looking just one token ahead (this is the LL(1) property).

Question 3 — Grammar Optimization for SQL Query Validation

Step 1: Where the redundancy comes from

A first-draft SQL grammar often repeats the same idea in several different non-terminals (e.g., separate rules for a column list with one column, two columns, three columns...) instead of one recursive rule, and it usually treats WHERE conditions with a single flat, ambiguous rule such as $\text{Condition} \rightarrow \text{Condition AND Condition} \mid \text{Condition OR Condition} \mid \text{ID op VALUE}$. That single rule is ambiguous for the same reason Question 1's expression grammar was: 'a=1 OR b=2 AND c=3' can be grouped two different ways, and only one respects the real precedence rule that AND binds tighter than OR.

Step 2: Optimized, layered CFG

SIMATS ENGINEERING ASSESSMENT TOOLS

```
Query      → 'SELECT' ColumnList 'FROM' ID WhereClause
ColumnList → '*' | ID ColumnTail
ColumnTail → ',' ID ColumnTail | ε

WhereClause → 'WHERE' OrExpr | ε
OrExpr      → AndExpr OrTail
OrTail      → 'OR' AndExpr OrTail | ε
AndExpr     → SimpleCond AndTail
AndTail     → 'AND' SimpleCond AndTail | ε
SimpleCond  → ID CompOp Value | '(' OrExpr ')'
CompOp      → '=' | '<' | '>' | '<=' | '>=' | '<>'
```

Step 3: How this removes redundancy and ambiguity

ColumnTail replaces what used to be several near-duplicate rules (one column, two columns, N columns) with a single recursive rule — this is the same trick used to describe repeating lists, and it collapses an unbounded family of productions into just two. On the WHERE side, splitting Condition into OrExpr (outer, lowest precedence) and AndExpr (inner, higher precedence) means AND sub-expressions are always fully grouped before an OR can combine them, exactly the same layering technique as Question 1's E/T/F split. 'a=1 OR b=2 AND c=3' can now only be parsed as a=1 OR (b=2 AND c=3).

Step 4: Why the validation tool benefits

- Fewer productions to maintain and test — one recursive rule instead of many similar-looking ones.
- A single, predictable parse tree per query, so the validator's error messages point to one unambiguous location in the query.
- Faster syntax analysis, since the parser table (or recursive-descent call structure) is smaller and has no ambiguous choice points to resolve.

Question 4 — CFG for a Subject–Verb–Object Chatbot

Step 1: Why the first grammar over-accepts sentences

A naive grammar such as Sentence $\rightarrow$ Word Word Word, where Word is any token in the vocabulary, accepts 'Students learn TOC' but also nonsense like 'TOC Students learn' or 'learn learn learn', because it never distinguishes what kind of word each position needs. The fix is not to loosen the grammar but to separate the vocabulary itself into grammatical categories (noun, verb) and only allow each category in the position where it grammatically belongs.

Step 2: Category-restricted CFG

```
Sentence → Subject Verb Object
Subject  → Noun
Object   → Noun
```

SIMATS ENGINEERING ASSESSMENT TOOLS

Noun	→ 'Students' 'TOC' 'chatbot' 'sentence'
Verb	→ 'learn' 'process' 'validates' 'understands'

For a richer chatbot this is extended with determiners and modifiers, still keeping each category closed: Subject → (Det) Noun, Det → 'the' | 'a', so 'the chatbot understands sentences' also parses correctly while 'chatbot the understands' still does not, because Det is only allowed before Noun, never after.

Step 3: How this improves validation accuracy

Because Verb and Noun are disjoint terminal sets, a string like 'Students learn TOC' derives successfully only when a noun-class word occupies the Subject and Object slots and a verb-class word occupies the Verb slot, in that exact order. 'learn Students TOC' fails immediately because 'learn' cannot be derived from Subject → Noun. This is exactly how a real CFG-based grammar checker rejects ungrammatical input: not through extra logic bolted on afterwards, but because the illegal string has no derivation at all under the production rules.

Step 4: Practical note for the chatbot team

- Category-restricted grammars scale by growing the terminal lists (Noun, Verb, Det, Adjective) without changing the structural rules — new vocabulary doesn't introduce new ambiguity.
- This mirrors how real natural-language parsers start: a CFG skeleton ($S \rightarrow NP VP$) plus a lexicon that tags each word with its part of speech.
- Limitation worth noting: a pure CFG still can't catch semantic nonsense like 'TOC learns Students' (grammatically valid, semantically odd) — that needs meaning-level checks beyond syntax.

Question 5 — Parser for a Simplified HTML-like Markup Language

Step 1: Why nested tags need recursion, not just concatenation

The core requirement — every opening tag must have a matching closing tag, and tags may nest to any depth (e.g. `<div><p><b>text</b></p></div>`) — is precisely the kind of 'balanced bracket' structure that regular expressions cannot describe (this is the classic example used to show a language is not regular, provable with the pumping lemma), but that context-free grammars handle naturally through self-reference.

Step 2: Recursive CFG for nested elements

Document	→	ElementList
ElementList	→	Element ElementList ϵ
Element	→	'<' Tag '>' ElementList '</' Tag '>' Text
Tag	→	letter Tag letter
Text	→	char Text char

Step 3: How the recursion guarantees correct nesting

Element calls ElementList inside its own body, and ElementList can produce more Elements — so an Element can legally contain any number of further Elements, each of which can contain further Elements, and so on to

SIMATS ENGINEERING

ASSESSMENT TOOLS

any depth. Because the closing '`</Tag>`' only appears at the end of the same production that opened '`<Tag>`', a closing tag can never appear before all of its inner content has been fully closed first; the grammar makes 'improperly nested' strings like `<div><p></div></p>` underivable.

One subtlety worth flagging for full marks: the grammar above enforces balanced nesting structurally, but checking that the opening tag name and closing tag name are literally the same string (e.g. rejecting `<div>...</b>`) is technically outside what a single context-free grammar can enforce in full generality for an unbounded set of tag names — that requirement behaves like a 'copy language', which typically needs a context-sensitive check or a semantic action (e.g., a symbol-table/stack lookup during parsing) layered on top of the CFG. In practice, parser generators handle this by keeping the CFG for structure and adding a small semantic check for name-matching.

Step 4: Why this design is reliable for the web-editor team

- Balanced, arbitrarily deep nesting is captured with just five production rules — no fixed depth limit is hard-coded anywhere.
- A stack-based parser (or a simple recursive-descent parser, which uses the call stack automatically) can validate nesting in a single left-to-right pass.
- Separating structural validation (CFG) from name-matching validation (semantic check) keeps the grammar clean and keeps error messages precise — 'unclosed tag' vs 'mismatched tag name' can be reported separately.

Quick summary across all five case studies

- Ambiguity is fixed by layering non-terminals in order of precedence (Q1, Q3).
- Left recursion is fixed with the standard $A \rightarrow \beta A', A' \rightarrow \alpha A' \mid \epsilon$ rewrite, without changing the language (Q2).
- Redundant productions collapse into recursive list-style rules (Q3).
- Restricting terminal categories enforces correct word order and rejects invalid strings by construction (Q4).
- Self-referencing (recursive) productions are what let a CFG describe unbounded, correctly nested structure that a regular expression could never match (Q5).